

Buổi 01 (3 tiết) – Tổng quan Mobile đa nền tảng & React Native

Bài mở đầu: Tổng quan về lập trình ứng dụng mobile đa nền tảng

Giới thiệu về lập trình ứng dụng di động

Định nghĩa về ứng dụng di động và sự phát triển của nền tảng di động:

Phát triển ứng dụng di động đa nền tảng bao gồm việc tạo ra các ứng dụng tương thích với nhiều hệ điều hành di động, sử dụng một cơ sở mã duy nhất. Cách tiếp cận này trái ngược với phát triển ứng dụng gốc, trong đó mã chuyên dụng được viết cho từng nền tảng cụ thể. Bằng cách tận dụng các công nghệ phát triển đa nền tảng, các nhà phát triển có thể xây dựng các ứng dụng di động cung cấp chức năng và giao diện nhất quán trên các thiết bị đa dạng, giảm đáng kể thời gian và chi phí phát triển. Ngoài ra, tích hợp mã gốc trong các ứng dụng đa nền tảng là một thực tiễn phổ biến để tối ưu hóa hiệu suất và đạt được trải nghiệm giống như gốc hơn trên mỗi nền tảng.

Sự hấp dẫn của việc phát triển ứng dụng đa nền tảng nằm ở khả năng hợp lý hóa quá trình tạo ứng dụng. Thay vì quản lý các dự án phát triển riêng cho iOS và các thiết bị Android, các nhà phát triển có thể tập trung vào một dự án duy nhất, đảm bảo triển khai nhanh hơn và bảo trì dễ dàng hơn. Hiệu quả này làm cho việc phát triển ứng dụng đa nền tảng trở thành một lựa chọn hấp dẫn cho các doanh nghiệp muốn tiếp cận đối tượng rộng hơn mà không ảnh hưởng đến chất lượng hoặc gặp phải chi phí phát triển tăng cao.

Sự cần thiết của việc phát triển ứng dụng di động đa nền tảng.

Sự phát triển liên tục của bối cảnh kỹ thuật số đã nhấn mạnh tầm quan trọng ngày càng tăng của các giải pháp đa nền tảng trong lĩnh vực phát triển ứng dụng di động. Khi các doanh nghiệp cố gắng tiếp cận người dùng trên cả nền tảng iOS và Android, khả năng tạo ra một ứng dụng duy nhất chạy trơn tru trên các thiết bị khác nhau trở nên cần thiết. Cách tiếp cận này, thu hẹp khoảng cách giữa phát triển gốc và đa nền tảng, cho phép sử dụng tài nguyên hiệu quả bằng cách cho phép chia sẻ mã nguồn trên các nền tảng, từ đó hợp lý hóa quá trình phát triển.

Hơn nữa, việc áp dụng các chiến lược phát triển ứng dụng đa nền tảng phản ánh xu hướng rộng lớn hơn của ngành hướng tới tính linh hoạt và khả năng mở rộng. Bằng cách tập trung vào việc tạo ra các trải nghiệm giống như bản địa với hiệu quả của các công cụ đa nền tảng, các nhà phát triển có thể đáp ứng mong đợi của người dùng về chức năng liền mạch và hiệu suất cao trên mọi thiết bị. Sự nhấn mạnh chiến lược này đối với các giải pháp đa nền tảng không chỉ là một phản ứng đối với nhu cầu thị trường hiện tại mà còn là một cách tiếp cận hướng tới tương lai dự đoán tương lai của việc phát triển ứng dụng di động.

Các lợi ích và thách thức của việc phát triển ứng dụng di động đa nền tảng.

1. Phát triển nhanh hơn

Với các giải pháp đa nền tảng, các nhà phát triển chủ yếu làm việc với một cơ sở mã để phát triển ứng dụng cho cả iOS và Android, loại bỏ nhu cầu xây dựng các dự án riêng biệt. Mọi thứ đều ở một nơi. Các ứng dụng đa nền tảng được xây dựng dưới dạng các dự án đơn lẻ nhưng hỗ trợ các thiết bị khác nhau. Có thể tái sử dụng một phần lớn mã của ứng dụng giữa các nền tảng và đó là lý do tại sao quá trình phát triển nhanh hơn nhiều.

2. Chi phí thấp hơn

So với các ứng dụng gốc, [một ứng dụng đa nền tảng có thể rẻ hơn khoảng 30%](#) hơn là xây dựng các ứng dụng iOS và Android riêng biệt. Bạn chỉ có một dự án cho cả iOS và Android, và các khối mã lớn và các tài sản khác có thể được sử dụng lại giữa các nền tảng. Bằng cách này, việc phát triển đa nền tảng nhanh hơn, do đó, chi phí tổng thể cho mỗi nền tảng ít hơn so với [phát triển các ứng dụng di động gốc](#) .

3. Đối tượng rộng hơn

Nhiều chủ sở hữu ứng dụng phải đối mặt với một thách thức khi bắt đầu phát triển ứng dụng di động: tốt hơn là tạo ứng dụng Android hay iOS trước? Bất kể nền tảng nào có thị trường lớn hơn, nó luôn là một sự đánh đổi có thể tốn rất nhiều tiền. Bắt đầu với giải pháp đa nền tảng, bạn sẽ có được đối tượng rộng hơn nhiều ngay từ đầu - bạn có thể nhắm mục tiêu cả hai thị trường cùng một lúc. Bạn cũng có thể xây dựng cho một nền tảng trước để phát hành sản phẩm nhanh hơn và sau đó nhanh chóng lặp lại việc thêm hỗ trợ cho nền tảng thứ hai.

3. Tính nhất quán giữa các nền tảng

iOS và Android có một số khác biệt về điều hướng và thiết kế xây dựng. Trong phát triển đa nền tảng, hầu hết các khác biệt về giao diện người dùng phổ biến được xử lý theo mặc định. Trong [sự phát triển bản địa](#) , ngay cả trong một nhóm có khả năng giao tiếp tốt, có thể có một số khác biệt trong việc thực hiện các chức năng. **Trong phát triển đa nền tảng, tình huống này không xảy ra thường xuyên, vì các nền tảng chia sẻ cùng một cơ sở mã** . Nó cũng cho phép bạn xây dựng bản sắc thương hiệu mạch lạc hơn trong các ứng dụng trên cả hai nền tảng với ít nỗ lực hơn.

4. Mã có thể tái sử dụng

Đây là một trong những điều tuyệt vời nhất về các ứng dụng đa nền tảng - có thể tạo một cơ sở mã cho Android và iOS cùng một lúc. Trong các ứng dụng gốc, cần phải viết mã riêng biệt, thường được thực hiện bởi hai nhà phát triển hoặc nhóm khác nhau. Trong phát triển ứng dụng di động tùy chỉnh, toàn bộ cơ sở mã nằm ở một nơi. Điều này giúp tiết kiệm rất nhiều thời gian, bởi vì một nhà phát triển có thể xử lý đồng thời cả hai nền tảng.

Giới thiệu về React Native

React native là một công cụ giúp chúng ta lập trình đa nền tảng để tạo ra các ứng dụng trên môi trường native. Nó là một framework mã nguồn mở được phát triển bởi Facebook, cho phép bạn sử dụng Java script để phát triển phần mềm trên điện thoại di động Android và IOS. React native cũng giống như React vậy chúng sử dụng các native components thay vì các web components. Vì vậy để hiểu về cấu trúc của React native chúng ta cần phải có các kiến thức cơ bản với các khái niệm cơ bản của React như là JSX, components, props hay là state.

Trước khi đi vào các phần chính, chúng ta hãy thử điếm qua xem rằng React native có những điểm lợi và điểm yếu gì. Nó có đáng để chúng ta bỏ công sức ra để tìm hiểu hay không?

Tổng quan về React Native và lịch sử phát triển của nó.

React Native là một framework **mã nguồn mở** dùng để xây dựng **ứng dụng di động đa nền tảng** (Android, iOS) bằng cách sử dụng **JavaScript** và **React** (một thư viện giao diện do Meta phát triển). Thay vì render giao diện bằng HTML/CSS như React trên web, React Native sử dụng các **thành phần giao diện gốc (native components)** để hiển thị giao diện người dùng.

React Native hoạt động như thế nào?

- Code JavaScript của bạn được **chạy trong một JavaScript engine (JSC)**.
- React Native sử dụng “**bridge**” (cầu nối) để kết nối giữa JavaScript và mã native.
- UI được hiển thị bằng **native views**, chứ không phải WebView, nhờ đó mang lại trải nghiệm gần như ứng dụng native.

Lịch sử phát triển React Native

Mốc thời gian	Sự kiện quan trọng
2013	Bắt đầu là một dự án nội bộ tại Facebook (tên mã: Hackathon). Ý tưởng xuất phát từ nhu cầu viết ứng dụng cho iOS bằng React.
Tháng 1/2015	React Native được giới thiệu chính thức tại React.js Conf 2015 .
Tháng 3/2015	React Native được phát hành công khai cho iOS (trên GitHub, mã nguồn mở).
Tháng 9/2015	Hỗ trợ Android chính thức được bổ sung. Giờ đây có thể viết ứng dụng Android và iOS chỉ với một codebase.
2016 – 2018	React Native phát triển mạnh, được dùng bởi nhiều công ty lớn như Instagram, Airbnb, Walmart, Skype...
2019	Facebook tuyên bố khởi động kế hoạch tái cấu trúc React Native với dự án " The New Architecture ", cải tiến performance và module system.
2020	Ra mắt Hermes engine cho Android (và sau đó cho iOS) để tối ưu hiệu năng JavaScript.

Mốc thời gian	Sự kiện quan trọng
2022 nay	Các cải tiến lớn với: TurboModules, Fabric Renderer (thay thế render engine cũ), và nâng cấp hệ thống bridging (JS ↔ native) giúp hiệu năng tốt hơn.
2023+	React Native tiếp tục phát triển mạnh mẽ, cập nhật thường xuyên, cải thiện trải nghiệm lập trình viên và tương thích chặt chẽ hơn với hệ sinh thái React (React 18+).

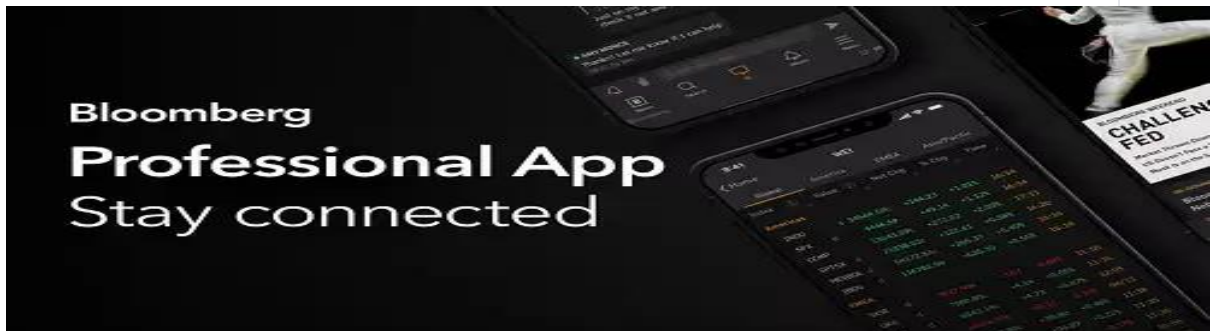
Sự khác biệt giữa ứng dụng native và ứng dụng React Native.

Tiêu chí	Ứng dụng Native	Ứng dụng React Native
Ngôn ngữ lập trình	Java/Kotlin (Android) Swift/Objective-C (iOS)	JavaScript (hoặc TypeScript), sử dụng thư viện React
Codebase	Mỗi nền tảng có một codebase riêng (Android & iOS viết riêng)	Một codebase duy nhất cho cả Android và iOS
Hiệu suất	Rất cao vì tương tác trực tiếp với nền tảng native	Cao, gần native, nhưng có thể chậm hơn trong các ứng dụng phức tạp vì dùng bridge (JS → native)
Truy cập API hệ thống (camera, GPS, sensor,...)	Truy cập trực tiếp, dễ dàng và tối ưu	Có thể dùng module có sẵn hoặc viết native module để gọi tới API hệ thống

Tiêu chí	Ứng dụng Native	Ứng dụng React Native
Tùy chỉnh UI nâng cao	Tùy biến toàn diện theo từng nền tảng	Dễ phát triển UI cơ bản, nhưng nếu cần hiệu ứng/phản hồi UI đặc thù của hệ điều hành, có thể phải viết native code
Tái sử dụng mã nguồn	Không tái sử dụng giữa các nền tảng	Có thể tái sử dụng 90%+ mã nguồn cho cả Android và iOS
Cộng đồng & thư viện	Tùy thuộc vào nền tảng (Android/iOS SDK)	Cộng đồng lớn, nhiều thư viện mã nguồn mở hỗ trợ đa nền tảng
Tốc độ phát triển ứng dụng	Chậm hơn vì cần 2 team hoặc 2 bộ mã	Nhanh hơn vì chỉ cần 1 team JS/React để làm cho cả 2 nền tảng
Cập nhật tính năng hệ điều hành mới	Được hỗ trợ ngay sau khi SDK cập nhật	Có thể bị chậm do phụ thuộc vào cộng đồng cập nhật module
Debug & test	Sử dụng Android Studio, Xcode với công cụ gốc	Dùng Chrome DevTools, Flipper, hoặc React Native Debugger
Phù hợp với ứng dụng nào?	Ứng dụng cần tối ưu hiệu năng cao, đồ họa phức tạp (game 3D, app video streaming)	Ứng dụng thông tin, mạng xã hội, thương mại điện tử, startup MVP, prototyping

Các ứng dụng nổi bật được xây dựng bằng React Native.

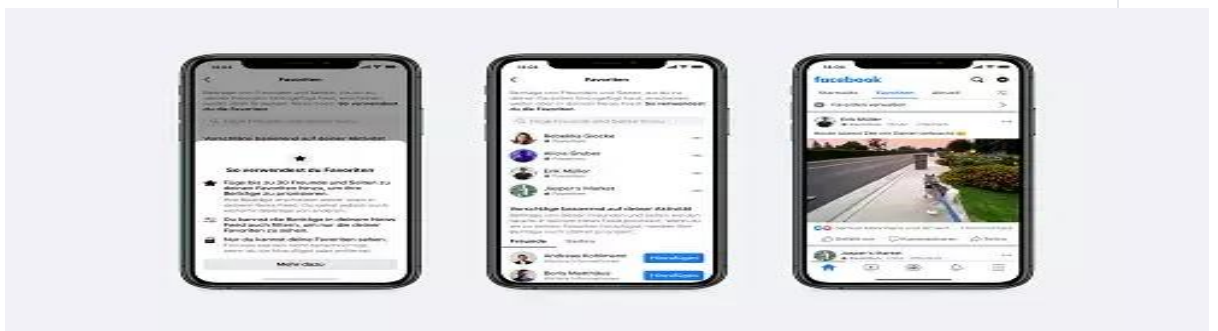
1.Bloomberg



Bloomberg đã sử dụng nó từ lâu vào năm 2016 khi ra mắt ứng dụng được cải tiến cho nền tảng iOS và Android. Công ty [đã sử dụng React Native](#) để giới thiệu các tính năng. Nội dung được cá nhân hóa, video và nguồn cấp dữ liệu trực tiếp đã được phát triển bằng React Native.

Trước React Native, các nhóm Bloomberg thường phát triển song song iOS và Android. Họ không biết rằng họ sẽ không thể tái sử dụng nhiều mã họ đã viết. Mô hình này thường trở nên tệ hại theo thời gian. Cuối cùng, nó sẽ dẫn đến việc làm việc lặp đi lặp lại và kém hiệu quả.

2.Facebook



Tại Meta, React và React Native không chỉ là công cụ; chúng là một phần không thể thiếu trong quá trình phát triển và đổi mới sản phẩm của chúng tôi. Những công

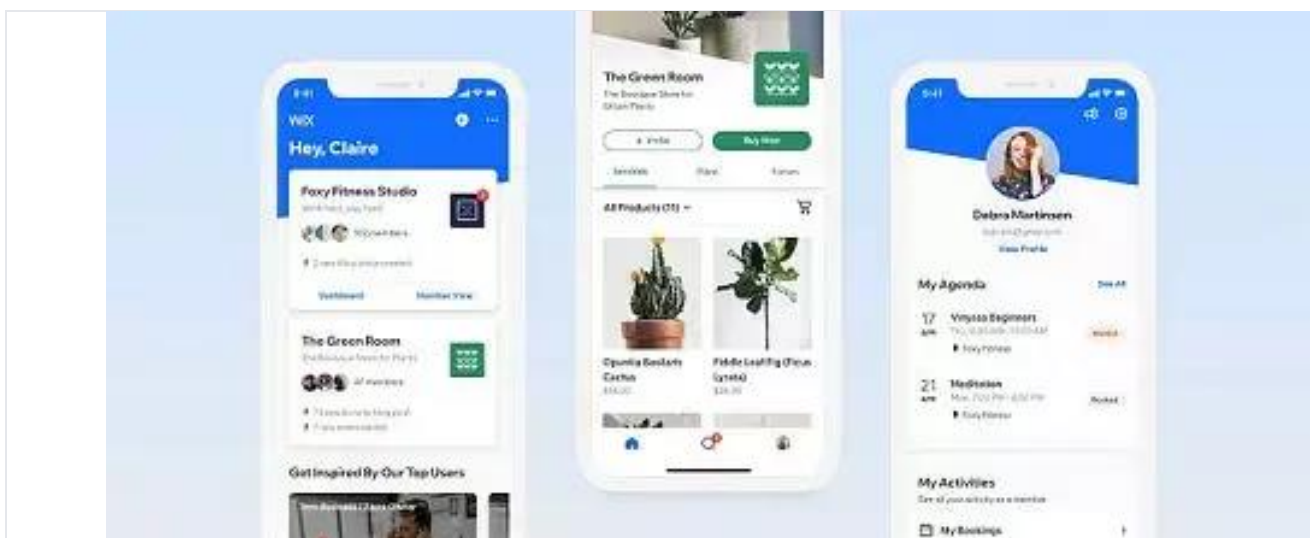
nghe này là nền tảng cho văn hóa kỹ thuật của chúng tôi, cho phép chúng tôi nhanh chóng xây dựng và triển khai các sản phẩm chất lượng cao.

Meta còn nhấn mạnh thêm những tiến bộ gần đây trong đó React và React Native đóng vai trò quan trọng trong việc thúc đẩy sự đổi mới.

Mặc dù phạm vi sử dụng React Native trong việc phát triển ứng dụng Facebook cốt lõi vẫn chưa được tiết lộ, nhưng có tài liệu ghi chép rõ ràng rằng Meta tận dụng React Native trên nhiều sản phẩm khác nhau.

Trang giới thiệu React Native đề cập rõ ràng đến việc triển khai trong các ứng dụng chủ lực, bao gồm Facebook, Instagram, Facebook Ads Manager, Meta Horizon và Messenger Desktop. Điều này chứng minh tính linh hoạt và đóng góp đáng kể của React Native vào hệ sinh thái sản phẩm của Meta.

Trình quản lý quảng cáo Facebook được [xây dựng hoàn toàn bằng React Native](#) , trong khi công ty sử dụng mã gốc và React Native JavaScript trong ứng dụng Nhóm Facebook của mình.



Nguồn hình ảnh: Medium

Wix là một trong những công ty [đầu tiên áp dụng công nghệ React Native](#) . Đây là một trình xây dựng trang web phổ biến cung cấp nhiều công cụ và mẫu dựng sẵn để xây dựng trang web mà không cần viết mã. Họ đã áp dụng công nghệ này vào

năm 2015, khi nhóm di động Wix còn nhỏ, chỉ có 4 thành viên. Họ quyết định triển khai React vì những lý do sau:

- Để làm cho ứng dụng di động của họ hiệu quả hơn
- Để phát triển ứng dụng trong thời gian ngắn hơn
- Để phát triển phần quan trọng của ứng dụng bằng cách sử dụng một mã duy nhất (nhóm chia sẻ hơn 95% cơ sở mã logic kinh doanh cho iOS và Android)
- Để xây dựng một ứng dụng có khả năng mở rộng

Với React, nhóm có thể phát triển một ứng dụng có hiệu suất cao trong thời gian ngắn, chỉ cần thêm một số thay đổi.

Walmart



Walmart là cửa hàng bán lẻ trực tuyến hàng đầu thế giới. Walmart cung cấp cho người dùng trang web và ứng dụng di động hàng đầu để mua sắm tại nhà.

Họ chuyển sang React Native để xây dựng một ứng dụng có khả năng mở rộng với số lượng người dùng điện thoại thông minh ngày càng tăng. Họ đã tận dụng React Native trong quá trình phát triển ứng dụng Android và iOS của mình.

Walmart đã chuyển sang React Native để tăng tốc độ phát triển. Ngoài ra, công ty đã quan sát thấy hiệu suất tốt hơn trong quá trình chuyển đổi trang và cảm giác gốc trên toàn bộ ứng dụng.



Instagram là một ví dụ phổ biến khác về ứng dụng chạy bằng React. Ứng dụng này có hơn 2 tỷ người dùng hoạt động hàng tháng, trở thành **nền tảng truyền thông xã hội phổ biến thứ 4** trên toàn cầu. Sau khi tích hợp React Native vào ứng dụng di động Facebook và Facebook Ads Manager, Meta đã sử dụng công nghệ do chính mình phát triển trong nền tảng thứ ba – Instagram.

Họ đã đưa nhiều tính năng khác nhau vào ứng dụng Instagram **bằng React Native**, như cải tiến giao diện chỉnh sửa hồ sơ hiện có, thêm tính năng thông báo

đây và nhóm phát triển sản phẩm được sử dụng để xây dựng nhiều thành phần cốt lõi khác nhau.

1.1 Kiến thức cơ bản về JavaScript và React

Tóm tắt về ngôn ngữ lập trình JavaScript và tại sao nó được sử dụng trong React Native.

JavaScript là gì?

- **JavaScript (JS)** là một **ngôn ngữ lập trình kịch bản** (scripting language) phổ biến nhất trên web.
- Ban đầu được tạo ra để **tương tác với HTML/CSS trên trình duyệt**, nhưng hiện nay đã phát triển thành ngôn ngữ đa nền tảng.

Đặc điểm chính:

- **Ngôn ngữ thông dịch**, chạy trực tiếp trong trình duyệt hoặc môi trường như Node.js.
- **Hướng đối tượng** (object-oriented), **bất đồng bộ** (asynchronous), hỗ trợ **event-driven programming**.
- Được dùng để phát triển: website, ứng dụng web, ứng dụng desktop, server-side, và cả ứng dụng di động (qua React Native).

Tại sao React Native dùng JavaScript?

Lý do	Giải thích
Phổ biến và dễ tiếp cận	JavaScript là một trong những ngôn ngữ được dùng rộng rãi nhất thế giới. Lập trình viên dễ học, dễ tiếp cận.

Lý do	Giải thích
Tích hợp tốt với React	React Native được xây dựng dựa trên React – một thư viện giao diện viết bằng JavaScript.
Viết 1 lần, chạy đa nền tảng	Với JavaScript, bạn có thể viết code cho cả Android và iOS cùng lúc.
Hỗ trợ hot-reloading / live-reload	JavaScript dễ hỗ trợ các tính năng như cập nhật UI mà không cần build lại ứng dụng.
Nhiều thư viện và cộng đồng hỗ trợ	Có sẵn hàng nghìn thư viện JavaScript dùng được ngay trong React Native (navigation, state, API, animation, v.v.).
Kết hợp được với native module	Dù dùng JavaScript, React Native vẫn có thể gọi code native khi cần thông qua "bridge".

Giới thiệu về thư viện React và cách nó hoạt động.

React là một **thư viện JavaScript mã nguồn mở** do **Meta (Facebook)** phát triển, dùng để xây dựng **giao diện người dùng (UI)**, đặc biệt là các **ứng dụng web đơn trang (SPA – Single Page Applications)**.

- Ra mắt lần đầu vào năm **2013**.
- React **không phải framework** (như Angular hay Vue) mà chỉ tập trung vào **"View"** trong mô hình **MVC (Model - View - Controller)**.

Tính năng	Mô tả
Component-based	Giao diện được chia nhỏ thành các component (thành phần) có thể tái sử dụng, độc lập.

Tính năng	Mô tả
JSX (JavaScript XML)	Cú pháp mở rộng giúp bạn viết mã HTML trong JavaScript một cách tự nhiên và dễ hiểu.
Virtual DOM	React sử dụng DOM ảo để so sánh (diffing) và cập nhật tối ưu phần thay đổi trên giao diện.
Unidirectional Data Flow	Dữ liệu chảy theo một chiều từ component cha xuống component con, dễ kiểm soát và debug.
Declarative	Bạn chỉ cần khai báo giao diện cần hiển thị, React lo phần hiển thị thực tế ra DOM.

React hoạt động như thế nào?

1. **Component** nhận **props** (dữ liệu đầu vào) và **state** (dữ liệu nội bộ).
2. React sử dụng **JSX** để mô tả giao diện cần hiển thị.
3. Khi có thay đổi về dữ liệu, React sẽ:
 - Cập nhật **Virtual DOM**.
 - So sánh Virtual DOM mới và cũ.
 - Tìm ra sự khác biệt (diffing).
 - Cập nhật **chính xác phần giao diện thay đổi** trong DOM thật (DOM gốc), giúp hiệu năng cao.

🔗 Cách hoạt động này gọi là "**reconciliation**" – quá trình tối ưu hóa việc cập nhật UI.

So sánh giữa lập trình React và lập trình React Native.

React (Web) vs. React Native (Mobile)

Tiêu chí	React	React Native
Mục đích sử dụng	Xây dựng giao diện ứng dụng web (SPA)	Xây dựng ứng dụng di động native cho Android & iOS
Nền tảng chạy	Trình duyệt web (Chrome, Firefox, Edge,...)	Thiết bị di động (Android, iOS)
Công nghệ render UI	HTML + CSS + DOM	Native components (View, Text, Image,...)
Ngôn ngữ giao diện	JSX + HTML + CSS	JSX + Stylesheet (JS dạng CSS)
Thẻ giao diện	Dùng các thẻ HTML: <div>, , <h1>, <button>, v.v.	Dùng các component tương tự HTML: <View>, <Text>, <Button>, v.v.
Thư viện CSS	Sử dụng CSS, SCSS, styled-components,...	Không dùng CSS thuần, dùng StyleSheet.create() hoặc styled-components/native
Thư viện điều hướng	react-router-dom	@react-navigation/native
Truy cập API thiết bị (camera, GPS, v.v.)	Hạn chế, phải dùng Web APIs nếu được hỗ trợ	Truy cập sâu vào phần cứng qua native modules

Tiêu chí	React	React Native
Xây dựng ứng dụng	Dễ dàng test và build, không cần compile native	Cần Android Studio / Xcode để build ứng dụng cho từng nền tảng
Triển khai sản phẩm	Deploy lên web server, hosting tĩnh, Vercel, Netlify,...	Đóng gói thành app .apk (Android), .ipa (iOS), deploy lên Play Store / App Store
Hot reload / Fast refresh	Có	Có
Hiệu suất	Rất nhanh trên trình duyệt	Gần như native (tốt hơn hybrid như Cordova)
Khả năng mở rộng	Tuyệt vời cho ứng dụng web	Tuyệt vời cho ứng dụng di động

1.2 Môi trường phát triển cho React Native

Các công cụ cần thiết để phát triển ứng dụng React Native trên Windows, macOS và Linux.

Yêu cầu chung (mọi hệ điều hành)

Công cụ	Mô tả
Node.js	Nền tảng JavaScript để chạy React Native CLI và quản lý gói npm/yarn
npm hoặc Yarn	Trình quản lý gói để cài thư viện
Java JDK (>= 11)	Cần thiết để build app Android
Android Studio	Bao gồm SDK Manager, AVD Emulator, build tools
Visual Studio Code (hoặc IDE khác)	Môi trường lập trình phổ biến cho React Native
React Native CLI hoặc Expo CLI	Tùy chọn cách khởi tạo và quản lý dự án

📖 Windows

Thành phần	Ghi chú
 Node.js	https://nodejs.org
 Android Studio	Dùng để build app Android, emulator
 Không build được iOS	iOS chỉ build được trên macOS
 React Native CLI hoặc Expo CLI	Expo dễ dùng hơn nếu không cần native module
 Visual Studio Code	Hỗ trợ plugin React Native Tools

macOS

Thành phần	Ghi chú
 Node.js	Qua Homebrew hoặc tải trực tiếp
 Android Studio	Để build Android
 Xcode	Bắt buộc để build và chạy ứng dụng iOS
 CocoaPods	Cài bằng <code>sudo gem install cocoapods</code> , để quản lý thư viện iOS
 React Native CLI / Expo CLI	Cả hai đều dùng được
 Visual Studio Code	IDE phổ biến

Linux (Ubuntu, Debian, v.v.)

Thành phần	Ghi chú
 Node.js	Qua <code>nvm</code> hoặc <code>apt</code>
 Android Studio	Build và test app Android
 Không hỗ trợ build iOS	Apple chỉ cho phép build iOS trên macOS
 React Native CLI / Expo CLI	Expo giúp đơn giản hoá build và test

Thành phần	Ghi chú
 Visual Studio Code	Có thể cài dễ dàng qua snap hoặc apt

Tùy chọn: React Native CLI vs Expo

Công cụ	Ưu điểm	Nhược điểm
React Native CLI	Toàn quyền kiểm soát, dùng được native module	Cài đặt phức tạp hơn
Expo CLI	Cài nhanh, test dễ, không cần Android Studio/Xcode ban đầu	Hạn chế khi cần native code tùy chỉnh

Cài đặt nhanh (ví dụ cho macOS):

bash

CopyEdit

1. Cài Node.js (qua Homebrew)

brew install node

2. Cài React Native CLI

npm install -g react-native-cli

3. Cài Android Studio & Xcode thủ công (tải từ web/App Store)

4. Khởi tạo dự án

```
npx react-native init MyApp
```

```
cd MyApp
```

```
npx react-native run-android # hoặc run-ios nếu dùng macOS
```

Các bước cài đặt và cấu hình môi trường phát triển.

1. Cài đặt Node.js

- Tải tại: <https://nodejs.org>
- Chọn phiên bản LTS (Long Term Support)
- Node.js sẽ cài sẵn npm – trình quản lý gói

2. Cài Java JDK 11+

- Tải tại: <https://www.oracle.com/java/technologies/javase-jdk11-downloads.html>
- Hoặc dùng bản mã nguồn mở như [Adoptium Temurin](#)

Sau khi cài, thêm đường dẫn JDK vào Environment Variables (biến môi trường):

- Biến hệ thống: JAVA_HOME → trỏ đến thư mục cài JDK
- Cập nhật biến Path → thêm %JAVA_HOME%\bin

3. Cài Android Studio

- Tải tại: <https://developer.android.com/studio>
- Trong quá trình cài, **chọn cài thêm Android SDK, SDK Platform, AVD (emulator)**

Cấu hình biến môi trường:

- Tạo biến:
 - ANDROID_HOME → trỏ đến SDK (mặc định: C:\Users\<user>\AppData\Local\Android\Sdk)
- Thêm vào Path:

```
plaintext
```

```
CopyEdit
```

```
%ANDROID_HOME%\platform-tools
```

```
%ANDROID_HOME%\emulator
```

Bạn có thể kiểm tra với lệnh `adb --version` sau khi thiết lập.

4. Cài React Native CLI

```
bash
```

```
CopyEdit
```

```
npm install -g react-native-cli
```

5. Khởi tạo và chạy dự án React Native

```
bash
```

```
CopyEdit
```

```
npx react-native init MyApp
```

```
cd MyApp
```

```
npx react-native run-android
```

Lưu ý: Phải mở **Android Emulator** hoặc kết nối thiết bị Android trước.

II. Cài đặt với Expo CLI (Dễ dàng, không cần Android Studio ban đầu)

1. Cài Node.js (giống bước trên)

2. Cài Expo CLI

```
npm install -g expo-cli
```

3. Khởi tạo ứng dụng Expo

```
expo init MyExpoApp
```

```
cd MyExpoApp
```

```
npm start # hoặc: expo start
```

Mở ứng dụng **Expo Go** trên điện thoại Android/iOS để quét QR code và chạy app ngay.

1.3 Tổng kết

Tóm tắt những điểm chính và ý nghĩa của chương.

Đề xuất cho các chủ đề sẽ được thảo luận trong các chương tiếp theo.

